

Softwarearchitektur – StudyQuest

Titel des Dokuments:

Softwarearchitektur – StudyQuest

Projektname:

StudyQuest – Gamifizierte Lern- und Notenverwaltungs-Web-App

Modul:

Software Engineering I – Praxis

Projektzeitraum:

Wintersemester 2025 / 2026

Version:

1.1

Datum:

26. Februar 2026

Author:innen:

- Michael Steer (Scrum Master)
- Luke Engehardt (Product Owner)
- Giuliana Carrano (Developer)
- Paul Strasser (Developer)
- Roman Faber (Developer)

Betreuer / Prüfer:

Sascha Wanninger

Freigabe durch autorisierte Person:

Michael Steer (Scrum Master)

Changelog

Version	Datum	Autor	Änderungsbesch
---------	-------	-------	----------------

0.1	12.10.2025	M. Steer	Erste 3-Schichten-Arch skizziert
0.2	15.10.2025	L. Engehardt	Modulübersicht und Verantwortlichei definiert
1.0	17.10.2025	M. Steer	Erstfassung der Softwarearchitekt erstellt
1.1	24.10.2025	M. Steer	Sicherheitsaspek und Observer-Pattern ergänzt, Finalversion zur Abgabe vorbereitet

Distribution List

Name		
Sascha Wanninger		

Michael Steer		
Luke Engehardt		
Giuliana Carrano		
Paul Strasser		
Roman Faber		

Einleitung

StudyQuest ist eine gamifizierte Web-App zur Lern- und Notenverwaltung. Die Implementierung basiert auf einem einfachen Client-Side-Stack (HTML, CSS und JavaScript) und verwendet den Browser-`LocalStorage` als Persistenzschicht. Das System wurde gemäß dem **3-Schichten-Modell** entwickelt, bestehend aus einer Präsentations-, einer Anwendungs- und einer Datenebene.

Hauptmodule & Interaktionen

Die Anwendung folgt einem dreischichtigen Architektur-Pattern. Die Präsentations- und Anwendungs-Logik laufen komplett im Browser. Es gibt kein

separates Backend. Die nachstehende Tabelle fasst die wichtigsten Module zusammen und ordnet sie der jeweiligen Schicht zu:

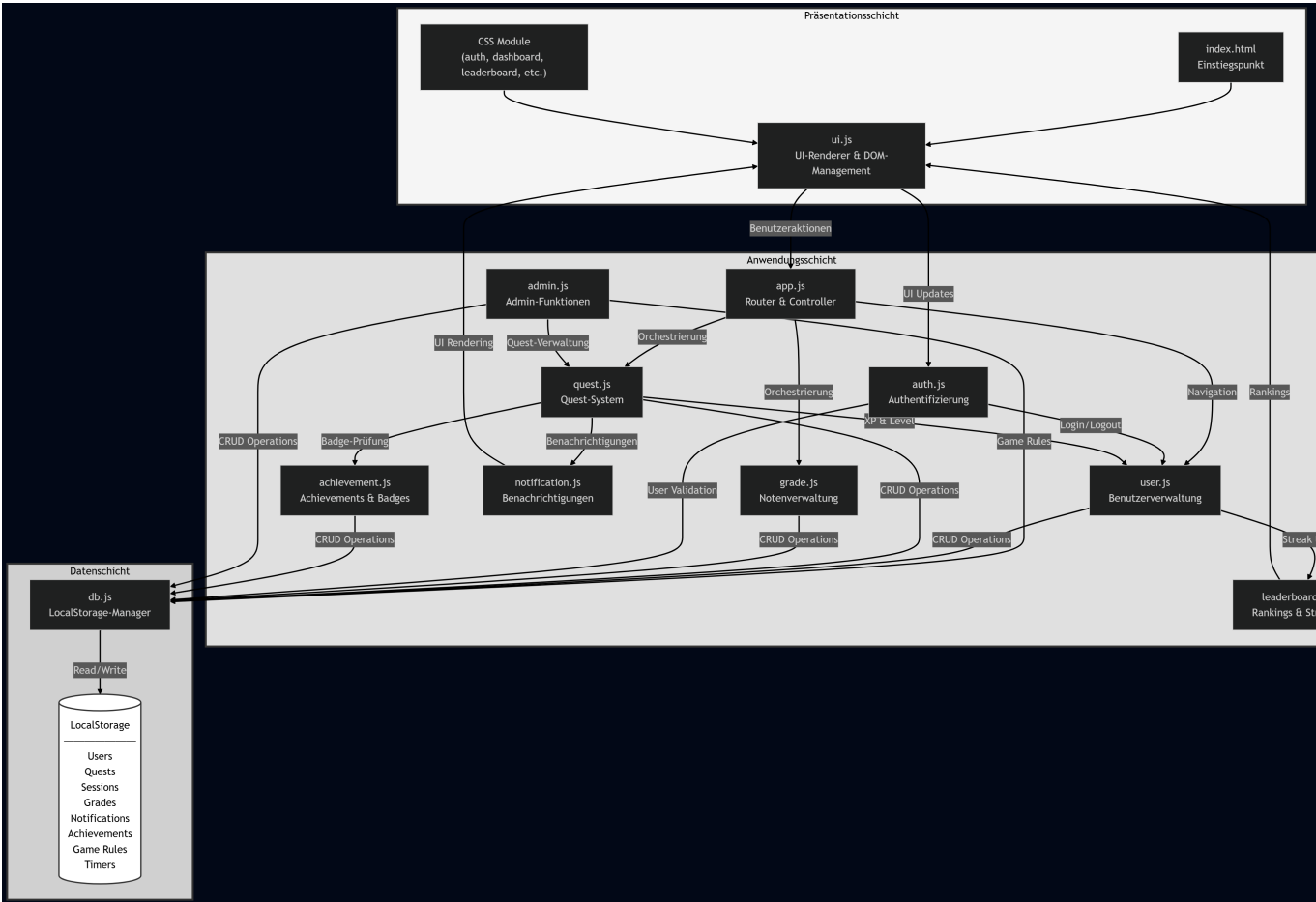
Schicht/Modul		A Erläuterung
Präsentationsschicht		Die Präsentationsschicht besteht aus <code>index.html</code> , diversen CSS-Dateien und <code>ui.js</code> . Sie liefert die Benutzeroberfläche, rendert Seiten dynamisch und leitet Benutzeraktionen an die Anwendungsschicht weiter.
<code>index.html</code>		Bindet CSS- und Einstiegspunkt JS-Module ein und definiert Platzhalter der Single-Page-App für das UI.
<code>css/</code>		Enthält Stildefinitionen für Authentifizierung, Styling Dashboard, Leaderboard, Admin-Bereiche usw.
<code>ui.js</code>		Verwaltet das DOM, zeigt Seiten (Login, Dashboard, UI-Übersicht, Admin-Panel usw.) an und reagiert auf Klicks.

Anwendungsschicht		Hier liegen sämtliche Geschäftslogik-Module. Die Module kommunizieren über gemeinsame Datenstrukturen und rufen sich gegenseitig auf.
app.js	Router & Controller	Zentrale Steuerung der Single-Page-App. Lädt Benutzer-Sessions, initialisiert die Datenbank, leitet Navigation (Dashboard, Quests, Leaderboard, Admin) und koordiniert andere Module.
auth.js	Authentifizierung	Handhabt Registrierung, Login/Logout sowie die Darstellung der Auth-Formulare. Ruft bei Erfolg user.js und app.js auf.
user.js	Benutzerverwaltung	Enthält das User-Modell (u. a. Level, XP, Rollen). Prüft Credentials, verwaltet die aktuelle Session und implementiert Rollen- und Streak-Logik.

<code>quest.js</code>		Startet und beendet Quests, berechnet XP-Belohnungen, legt Quest-System neue Lern-Sessions an und aktualisiert die Benutzer-Historie.
<code>grade.js</code>		CRUD-Operationen für Noten, unterstützt Notenverwaltung CSV-Import/Export und Tabellenansicht.
<code>notification.js</code>		Erzeugt Toasts und Bell-Notifications bei Quest-Abgeschlossen, Level-Ups und Achievements.
<code>achievement.js</code>		Prüft Bedingungen Achievements (Anzahl Quests, & Level, Geschwindigkeit) und Badges vergibt Badges.
<code>leaderboard.js</code>		Berechnet Rankings nach XP, Level, & erledigten Quests und Tages-Streaks, zeigt Top-10-Tabellen an.
<code>admin.js</code>		Ermöglicht das Erstellen/Ändern/Löschen von Quests, das Konfigurieren der Admin-Funktionen der Spielregeln sowie die Benutzer- und Rollenverwaltung.

Datenschicht	Ein einziges Modul, <code>db.js</code> , kapselt sämtliche Speicherzugriffe.
<code>db.js</code>	Verwaltet den Zugriff auf LocalStorage (Users, Quests, Sessions, Timer, Grades, Notifications, LocalStorage-Manager, Game Rules), initialisiert Default-Werte und bietet CRUD-Funktionen für jede Entität.

Interaktionsdiagramm



Die Pfeile zeigen den Kontroll- und Datenfluss: Die Präsentationsschicht ruft Funktionen der Anwendungs-Schicht auf (z. B. beim Abschließen einer Quest),

diese wiederum lesen/schreiben Daten über die Persistenzschicht. Umgekehrt sendet die Datenbank keine direkten Events an die Anwendung, sondern die Business-Logik zieht Daten bei Bedarf.

Abhängigkeiten zwischen Analyse-Klassen und Technologien

Das Analyseklassenmodell (siehe Grobdesign) definiert die wesentlichen Domänenklassen **User**, **Quest**, **LearningSession**, **Grade**, **Achievement**, **Notification**, **GameRule** und **Timer**. In der Implementierung werden diese Klassen größtenteils als JavaScript-Objekte und Collections in LocalStorage realisiert, einige werden durch Module gekapselt:

- **User** – Repräsentiert einen Benutzer mit Eigenschaften wie `id`, `email`, `name`, `password_hash`, `level`, `xp` (aktuelles Level-XP), `total_xp_earned` (kumulativ), `is_admin` (Boolean statt role), `current_streak`, `best_streak`, `last_activity_date`. Geschäftslogik-Methoden (z. B. `authenticate()`, `updateStreak()`, `isAdmin()`) befinden sich in `user.js`. Persistiert wird das Objekt in `users`-Store im LocalStorage.
- **Quest** – Definiert Lernaufgaben (`id`, `title`, `description`, `difficulty`, `xp_reward`, `status`). Funktionen wie `startQuest()`, `startTimer()`, `stopTimer()` (nicht `completeQuest()` - das ist implizit bei Timerend) und XP-Berechnung liegen in `quest.js`, die Instanzen werden im `quests`-Store des LocalStorage verwaltet.
- **LearningSession** – Audit-Trail für Quest-Abschlüsse, enthält `id`, `user_id`, `quest_id`, `xp_earned`, `duration_seconds`, `completed_at`. Sessions werden über `quest.js` (bei `stopTimer()`) und `db.js` erzeugt und gespeichert.

- **Grade** – Enthält Noten (`id`, `user_id`, `module_name` (nicht `subject`), `grade_value`, `semester`, `created_at`) und wird durch `grade.js` verwaltet. Persistiert im `grades`-Store, Import/Export via CSV wird vom Modul unterstützt.
- **Achievement** – Repräsentiert freischaltbare Badges, `achievement.js` definiert die Bedingungen und pflegt eine Liste freigeschalteter Badges im `achievements`-Store.
- **Notification** – Temporäre Ereignisse, die dem User angezeigt werden, gespeichert im `notifications`-Store und von `notification.js` angezeigt.
- **GameRule** – Konfigurierbare Spielparameter wie `xp_per_minute_timer` (Zeitzuschlag pro Minute Timer), `level_threshold` (XP pro Level-Aufstieg), `max_level`. `admin.js` erlaubt Admins, die Regeln zu bearbeiten, die Werte werden im `game_rules`-Store persistiert.
- **Timer** – Wird für laufende Quests benötigt, speichert `id`, `user_id`, `quest_id`, `start_time`, `end_time`, `duration_seconds`, `active` (Boolean) im `timers`-Store. `quest.js` (Funktionen `startTimer()`, `stopTimer()`) und `ui.js` koordinieren das Timer-Display und die XP-Bonus-Berechnung.

Framework-Abhängigkeiten: Die Anwendung nutzt kein externes Frontend- oder Backend-Framework, alle Module sind in Vanilla-JavaScript geschrieben. Als persistente Technologie dient der Browser-LocalStorage. Die einzige

„Framework-Abhängigkeit“ ist daher die Browser-API (DOM, LocalStorage). Diese beeinflusst das Klassenmodell wie folgt:

- Jedes Objekt muss eine serialisierbare Struktur besitzen (JSON), da LocalStorage nur Strings speichert. Deshalb haben alle Entitäten eindeutige `id`-Felder und werden als flache Objekte gespeichert.
- Passwörter werden im Prototyp mit einem vereinfachten Hash-Verfahren kodiert (`_hashPassword()`), was in einer produktiven Architektur durch eine kryptographische Hash-Funktion (z. B. bcrypt/Argon2) ersetzt werden sollte.
- Da keine Live-Datenbank existiert, gibt es keine konkurrierenden Zugriffe, Transaktionen werden in der Business-Logik implementiert (z. B. bei `stopTimer()` in `quest.js` wird Timerzeit erfasst, XP berechnet, Level aktualisiert und eine `LearningSession` angelegt in synchroner/atomarer Reihenfolge).

Verfeinerung des Analyse-Klassenmodells

Auf Basis der Implementierung wurden einige Anpassungen am Analyse-Klassenmodell vorgenommen:

1. **Eindeutige IDs und Persistenz-Attribute:** Alle Klassen besitzen ein `id`-Attribut, um Objekte im LocalStorage referenzieren zu können. Für Relationen (z. B. `user_id` in `LearningSession`, `quest_id` in `Timer`) werden

Fremdschlüssel als IDs gespeichert.

2. **Rollen und Rechte:** Das `User`-Objekt hat ein Feld `is_admin` (Boolean, nicht `role`) zur Unterscheidung zwischen normalen Benutzern und Administratoren. Der erste User, der sich registriert, erhält automatisch Admin-Status. `/admin.js` verwendet dieses Feld, um zu entscheiden, ob Admin-Funktionen angezeigt werden.
3. **Status-Felder:** `Quest` enthält ein `status`-Attribut (z. B. „available“, „active“, „completed“) und die IDs aktiver Quests werden im User-Objekt gespeichert (`active_quest_id`).
4. **Audit-Trail:** Die Klasse `LearningSession` wurde eingeführt, um jeden Quest-Abschluss mit Zeitstempel zu speichern. Diese Daten werden für Achievements, Leaderboard-Berechnungen und Streak-Updates genutzt.
5. **Timer als eigene Entität:** Obwohl Timer zunächst nur UI-Logik war, wird es als persistentes Objekt im `LocalStorage` gespeichert, damit das Zeit-Tracking nach Seiten-Reload fortgesetzt werden kann. Timer hält Verweise auf den User und die Quest und speichert Start- und Endzeit.

Referenzarchitektur

Die Anwendung orientiert sich an zwei bekannten Referenzarchitekturen:

3-Schichten-Architektur (Presentation – Application – Data): Die Schichten entkoppeln UI, Geschäftslogik und Persistenz. Dadurch können Anpassungen an der Oberfläche (z. B. neues CSS-Framework) ohne Änderungen der Logik erfolgen. Die Business-Schicht orchestriert Use Cases, während die Data-Schicht einzige Quelle der Wahrheit ist. In StudyQuest ist diese Architektur allerdings komplett client-seitig implementiert, da es keinen Server gibt.

Model–View–Controller (MVC) als Variationsmuster: Obwohl kein Framework wie Angular/React eingesetzt wird, lassen sich die Module grob dem MVC-Prinzip zuordnen. `ui.js` fungiert als View (Darstellung), die Module wie `user.js`, `quest.js` usw. bilden das Model (Domänenlogik), und `app.js` dient als Controller (Routing und Koordination). Dieses Muster erleichtert das Verständnis der Interaktionen und unterstützt die testbare Umsetzung der Use Cases.

Single-Page-Application (SPA) Architektur: Durch das Laden aller Module in `index.html` läuft die App vollständig im Browser. Das Routing zwischen Seiten wird von `app.js` gesteuert, ohne dass der Browser neu lädt. Die Persistenz in LocalStorage macht die Anwendung offline-fähig. In einer späteren Version könnte diese Schicht durch ein REST-Backend ersetzt werden.